

1804.10694v5 — formula report

56 inline formulas (0 shown: the ones that did not render), 7 display equations, 6 tables, 12 unrecovered image regions.

Display equations

Identifier	Page	Conf.	LaTeX source	Rendered	Scan image
1804.10694v5_E00007	008	■0.938 ●X +3	$\begin{gathered} \{b\ y(i,\ j,\ c)\ \rightarrow\ \text{by}\ (i\ 0,\ j\ 0,\ i\ 1,\ j\ 1,\ c):\ i\ 0=\text{floor}\ (i\ /\ 32)\ \\ \wedge\ i\ 1=i\ \% \ 32\ \wedge\ \backslash\ j\ 0=\text{floor}\ (j\ /\ 32)\ \\ \wedge\ j\ 1=j\ \% \ 32\ \wedge\ 0\ \leq\ i<N\ \wedge\ 0\ \leq\ j<N\backslash\} \\ \end{gathered}$	$\{by(i,j,c) \rightarrow \text{by}(i0,j0,i1,j1,c) : i0 = \text{floor}(i/32) \wedge i1 = i\%32 \wedge j0 = \text{floor}(j/32) \wedge j1 = j\%32 \wedge 0 \leq i < N \wedge 0 \leq j < N\}$	$\{by(i,j,c) \rightarrow by(i0,j0,i1,j1,c) : i0 = floor(i/32) \wedge i1 = i\%32 \wedge j0 = floor(j/32) \wedge j1 = j\%32 \wedge 0 \leq i < N \wedge 0 \leq j < N\}$
1804.10694v5_E00006	008	■0.976 ●W -1	$\begin{aligned} &\{b\ y(1,\ i\ \theta(g\ p\ u\ B),\ j\ \theta(g\ p\ u\ B),\ i\ 1(g\ p\ u\ T),\ j\ 1(g\ p\ u\ T),\ c)\ \rightarrow\ b\ y[c,\ i\ \theta\ * \ 32\ +\ \backslash\ \&\ i\ 1,\ j\ \theta\ * \ 32+j\ 1]:\ i\ \theta=\text{floor}\ (i\ /\ 32)\ \\ &\wedge\ j\ 0=\text{floor}\ (j\ /\ 32)\ \wedge\ i\ 1=i\ \% \ 32\ \\ &\wedge\ j\ 1=\backslash\ \&\ j\ \% \ 32\ \wedge\ 0\ \leq\ i<N-2\ \wedge\ 0\ \leq\ j<M-2\ \wedge\ 0\ \leq\ c<3\backslash\} \end{aligned}$	$\{by(1,i0(gpuB),j0(gpuB),i1(gpuT),j1(gpuT),c) \rightarrow by[c,i0 * 32 + i1,j0 * 32 + j1] : i0 = \text{floor}(i/32) \wedge j0 = \text{floor}(j/32) \wedge i1 = i\%32 \wedge j1 = j\%32 \wedge 0 \leq i < N - 2 \wedge 0 \leq j < M - 2 \wedge 0 \leq c < 3\}$	$\{by(1,i0(gpuB),j0(gpuB),i1(gpuT),j1(gpuT),c) \rightarrow by[c,i0 * 32 + i1,j0 * 32 + j1] : i0 = floor(i/32) \wedge j0 = floor(j/32) \wedge i1 = i\%32 \wedge j1 = j\%32 \wedge 0 \leq i < N - 2 \wedge 0 \leq j < M - 2 \wedge 0 \leq c < 3\}$
1804.10694v5_E00004	007	■0.990 ●X +1	$\begin{aligned} &\{b\ y(i,\ j,\ c):\ 0\ \leq\ i<N-2\ \wedge\ 0\ \leq\ j<M-2\ \wedge\ 0\ \leq\ c<3\backslash}: \backslash\ \&\ (b\ x(i,\ j,\ c)+b\ x(i+1,\ j,\ c)+b\ x(i+2,\ j,\ c))\ /\ 3\ \end{aligned}$	$\{by(i,j,c) : 0 \leq i < N - 2 \wedge 0 \leq j < M - 2 \wedge 0 \leq c < 3\} : (bx(i,j,c) + bx(i+1,j,c) + bx(i+2,j,c))/3$	$\{by(i,j,c) : 0 \leq i < N - 2 \wedge 0 \leq j < M - 2 \wedge 0 \leq c < 3\} : (bx(i,j,c) + bx(i+1,j,c) + bx(i+2,j,c))/3$
1804.10694v5_E00005	007	■0.994 ●N -1	$\{b\ y(1,\ i\ \theta(g\ p\ u\ B),\ j\ \theta(g\ p\ u\ B),\ i\ 1(g\ p\ u\ T),\ j\ 1(g\ p\ u\ T),\ c)\ \quad :\quad i\ \theta=$	$\{by(1,i0(gpuB),j0(gpuB),i1(gpuT),j1(gpuT),c) \quad : \quad i0 =$	$\{by(1,i0(gpuB),j0(gpuB),i1(gpuT),j1(gpuT),c) \quad : \quad i0 =$
1804.10694v5_E00001	007	■1.000 ●N +0	$\{(1,1)\ ;(2,1)\ ;(3,1)\ ;(1,2)\ ;(2,2)\ ;(3,2)\backslash\}$	$\{(1,1);(2,1);(3,1);(1,2);(2,2);(3,2)\}$	$\{(1,1);(2,1);(3,1);(1,2);(2,2);(3,2)\}$
1804.10694v5_E00002	007	■1.000 ●N +0	$\{S(i,\ j):\ 1\ \leq\ i\ \leq\ 3\ \wedge\ 1\ \leq\ j\ \leq\ 2\backslash\}$	$\{S(i,j) : 1 \leq i \leq 3 \wedge 1 \leq j \leq 2\}$	$\{S(i,j) : 1 \leq i \leq 3 \wedge 1 \leq j \leq 2\}$
1804.10694v5_E00003	007	■1.000 ●X +2	$\{S\ 1(i,\ j)\ \rightarrow\ S\ 2(i+2,\ j+2):\ 1\ \leq\ i\ \leq\ 3\ \wedge\ 1\ \leq\ j\ \leq\ 2\backslash\}$	$\{S1(i,j) \rightarrow S2(i+2,j+2) : 1 \leq i \leq 3 \wedge 1 \leq j \leq 2\}$	$\{S1(i,j) \rightarrow S2(i+2,j+2) : 1 \leq i \leq 3 \wedge 1 \leq j \leq 2\}$
Conf. MathPix confidence: ■≥0.9 ■0.5-0.9 ■<0.5 — no value LaTeX source MathPix’s reading, unchanged; Rendered the same reading, with a display environment (align/alignat/flalign/eqnarray/gather/multline/split) mapped to its in-math form so it can typeset in this cell — the two may legitimately differ Residual render vs scan (inkdrill): ●C component ●W weak ●S stable ●N noise ●K clean ●X crop overran its region ●not measured [refined: <i>basis</i>] beside an identifier: the LaTeX-source and Rendered cells both show a VERIFIED refinement, not MathPix’s own reading — <i>basis</i> names what verified it (e.g. ink, source). The original reading is unchanged in the model but is not the one drawn here.					

Tables

Identifier	Page	Conf.	LaTeX source	Rendered	Scan image																																																																																																																																																																																				
1804.10694v5_TAB_001	003	■ 1.000	$\begin{tabular}{l l l l l l l} \hline \text{Feature} & \text{Tiramisu} & \text{AlphaZ} & \text{PENCIL} & \text{Pluto} & \text{Halide} \\ \hline \text{CPU code generation} & \text{Yes} & \text{Yes} & \text{Yes} & \text{Yes} & \text{Yes} \\ \hline \text{GPU code generation} & \text{Yes} & \text{No} & \text{Yes} & \text{Yes} & \text{Yes} \\ \hline \text{Distributed CPU code generation} & \text{Yes} & \text{No} & \text{No} & \text{Yes} & \text{Yes} \\ \hline \text{Distributed GPU code generation} & \text{Yes} & \text{No} & \text{No} & \text{No} & \text{No} \\ \hline \text{Support all affine loop transformations} & \text{Yes} & \text{Yes} & \text{Yes} & \text{Yes} & \text{No} \\ \hline \text{Commands for loop transformations} & \text{Yes} & \text{Yes} & \text{No} & \text{No} & \text{Yes} \\ \hline \text{Commands for optimizing data accesses} & \text{Yes} & \text{Yes} & \text{No} & \text{No} & \text{Yes} \\ \hline \text{Commands for communication} & \text{Yes} & \text{No} & \text{No} & \text{No} & \text{No} \\ \hline \text{Commands for memory hierarchies} & \text{Yes} & \text{No} & \text{No} & \text{No} & \text{Limited} \\ \hline \text{Expressing cyclic data-flow graphs} & \text{Yes} & \text{Yes} & \text{Yes} & \text{Yes} & \text{No} \\ \hline \text{Non-rectangular iteration spaces} & \text{Yes} & \text{Yes} & \text{Yes} & \text{Yes} & \text{Limited} \\ \hline \text{Exact dependence analysis} & \text{Yes} & \text{Yes} & \text{Yes} & \text{Yes} & \text{No} \\ \hline \text{Compile-time set emptiness check} & \text{Yes} & \text{Yes} & \text{Yes} & \text{Yes} & \text{No} \\ \hline \text{Implement parametric tiling} & \text{No} & \text{Yes} & \text{No} & \text{No} & \text{Yes} \\ \hline \end{tabular}$	<table><tr><td>Feature</td><td>Tiramisu</td><td>AlphaZ</td><td>PENCIL</td><td>Pluto</td><td>Halide</td></tr><tr><td>CPU code generation</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td></tr><tr><td>GPU code generation</td><td>Yes</td><td>No</td><td>Yes</td><td>Yes</td><td>Yes</td></tr><tr><td>Distributed CPU code generation</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td><td>Yes</td></tr><tr><td>Distributed GPU code generation</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>No</td></tr><tr><td>Support all affine loop transformations</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Commands for loop transformations</td><td>Yes</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr><tr><td>Commands for optimizing data accesses</td><td>Yes</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr><tr><td>Commands for communication</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>No</td></tr><tr><td>Commands for memory hierarchies</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>Limited</td></tr><tr><td>Expressing cyclic data-flow graphs</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Non-rectangular iteration spaces</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Limited</td></tr><tr><td>Exact dependence analysis</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Compile-time set emptiness check</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Implement parametric tiling</td><td>No</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr></table>	Feature	Tiramisu	AlphaZ	PENCIL	Pluto	Halide	CPU code generation	Yes	Yes	Yes	Yes	Yes	GPU code generation	Yes	No	Yes	Yes	Yes	Distributed CPU code generation	Yes	No	No	Yes	Yes	Distributed GPU code generation	Yes	No	No	No	No	Support all affine loop transformations	Yes	Yes	Yes	Yes	No	Commands for loop transformations	Yes	Yes	No	No	Yes	Commands for optimizing data accesses	Yes	Yes	No	No	Yes	Commands for communication	Yes	No	No	No	No	Commands for memory hierarchies	Yes	No	No	No	Limited	Expressing cyclic data-flow graphs	Yes	Yes	Yes	Yes	No	Non-rectangular iteration spaces	Yes	Yes	Yes	Yes	Limited	Exact dependence analysis	Yes	Yes	Yes	Yes	No	Compile-time set emptiness check	Yes	Yes	Yes	Yes	No	Implement parametric tiling	No	Yes	No	No	Yes	<table><tr><td>Feature</td><td>Tiramisu</td><td>AlphaZ</td><td>PENCIL</td><td>Pluto</td><td>Halide</td></tr><tr><td>CPU code generation</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td></tr><tr><td>GPU code generation</td><td>Yes</td><td>No</td><td>Yes</td><td>Yes</td><td>Yes</td></tr><tr><td>Distributed CPU code generation</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td><td>Yes</td></tr><tr><td>Distributed GPU code generation</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>No</td></tr><tr><td>Support all affine loop transformations</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Commands for loop transformations</td><td>Yes</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr><tr><td>Commands for optimizing data accesses</td><td>Yes</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr><tr><td>Commands for communication</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>No</td></tr><tr><td>Commands for memory hierarchies</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>Limited</td></tr><tr><td>Expressing cyclic data-flow graphs</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Non-rectangular iteration spaces</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Limited</td></tr><tr><td>Exact dependence analysis</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Compile-time set emptiness check</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Implement parametric tiling</td><td>No</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr></table>	Feature	Tiramisu	AlphaZ	PENCIL	Pluto	Halide	CPU code generation	Yes	Yes	Yes	Yes	Yes	GPU code generation	Yes	No	Yes	Yes	Yes	Distributed CPU code generation	Yes	No	No	Yes	Yes	Distributed GPU code generation	Yes	No	No	No	No	Support all affine loop transformations	Yes	Yes	Yes	Yes	No	Commands for loop transformations	Yes	Yes	No	No	Yes	Commands for optimizing data accesses	Yes	Yes	No	No	Yes	Commands for communication	Yes	No	No	No	No	Commands for memory hierarchies	Yes	No	No	No	Limited	Expressing cyclic data-flow graphs	Yes	Yes	Yes	Yes	No	Non-rectangular iteration spaces	Yes	Yes	Yes	Yes	Limited	Exact dependence analysis	Yes	Yes	Yes	Yes	No	Compile-time set emptiness check	Yes	Yes	Yes	Yes	No	Implement parametric tiling	No	Yes	No	No	Yes
Feature	Tiramisu	AlphaZ	PENCIL	Pluto	Halide																																																																																																																																																																																				
CPU code generation	Yes	Yes	Yes	Yes	Yes																																																																																																																																																																																				
GPU code generation	Yes	No	Yes	Yes	Yes																																																																																																																																																																																				
Distributed CPU code generation	Yes	No	No	Yes	Yes																																																																																																																																																																																				
Distributed GPU code generation	Yes	No	No	No	No																																																																																																																																																																																				
Support all affine loop transformations	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Commands for loop transformations	Yes	Yes	No	No	Yes																																																																																																																																																																																				
Commands for optimizing data accesses	Yes	Yes	No	No	Yes																																																																																																																																																																																				
Commands for communication	Yes	No	No	No	No																																																																																																																																																																																				
Commands for memory hierarchies	Yes	No	No	No	Limited																																																																																																																																																																																				
Expressing cyclic data-flow graphs	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Non-rectangular iteration spaces	Yes	Yes	Yes	Yes	Limited																																																																																																																																																																																				
Exact dependence analysis	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Compile-time set emptiness check	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Implement parametric tiling	No	Yes	No	No	Yes																																																																																																																																																																																				
Feature	Tiramisu	AlphaZ	PENCIL	Pluto	Halide																																																																																																																																																																																				
CPU code generation	Yes	Yes	Yes	Yes	Yes																																																																																																																																																																																				
GPU code generation	Yes	No	Yes	Yes	Yes																																																																																																																																																																																				
Distributed CPU code generation	Yes	No	No	Yes	Yes																																																																																																																																																																																				
Distributed GPU code generation	Yes	No	No	No	No																																																																																																																																																																																				
Support all affine loop transformations	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Commands for loop transformations	Yes	Yes	No	No	Yes																																																																																																																																																																																				
Commands for optimizing data accesses	Yes	Yes	No	No	Yes																																																																																																																																																																																				
Commands for communication	Yes	No	No	No	No																																																																																																																																																																																				
Commands for memory hierarchies	Yes	No	No	No	Limited																																																																																																																																																																																				
Expressing cyclic data-flow graphs	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Non-rectangular iteration spaces	Yes	Yes	Yes	Yes	Limited																																																																																																																																																																																				
Exact dependence analysis	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Compile-time set emptiness check	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Implement parametric tiling	No	Yes	No	No	Yes																																																																																																																																																																																				

Conf. MathPix confidence: ■ ≥ 0.9 ■ 0.5–0.9 ■ < 0.5 — no value

LaTeX source MathPix’s reading, unchanged; **Rendered** the same reading, with a display environment (align/alignat/flalign/eqnarray/gather/multline/split) mapped to its in-math form so it can typeset in this cell — the two may legitimately differ

Residual render vs scan (inkdrill): ● C component ● W weak ● S stable ● N noise ● K clean ● X crop overran its region ● not measured

[refined: *basis*] beside an identifier: the LaTeX-source and Rendered cells both show a VERIFIED refinement, not MathPix’s own reading — *basis* names what verified it (e.g. ink, source). The original reading is unchanged in the model but is not the one drawn here.

Identifier	Page	Conf.	LaTeX source	Rendered	Scan image																														
1804.10694v5_TAB_002	004	0.600	<pre>\begin{tabular}[t]{l l } \hline \multicolumn{2}{ c }{ }We assume that $\mathbf{C}$ and $\mathbf{P}$ are computations, $\mathbf{b}$ is a buffer i and j are loop iterators} \\ \hline \multicolumn{2}{ c }{ Commands for loop nest transformations} \\ \hline Command & Description \\ \hline C.tile(i,j,t1,t2, i0,j0,i1,j1) & Tile the loop levels $((\mathrm{i}, \mathrm{j}))$ of the computation C by $(t_1 \times t_2)$. The names of the new loop levels are (i_0, j_0, i_1, j_1) where i_0 is the outermost loop level and j_1 is the innermost. \\ \hline C.interchange(i, j) & Interchange the i and j loop levels of C. \\ \hline C.shift(i, s) & Loop shifting (shift the loop level i by s iterations). \\ \hline C.split(i, s, i_0, i_1) & Split the loop level i by s. (i_0, i_1) are the new loop levels. \\ \hline P.compute_at(C, j) & Compute the computation P in the loop nest of C at loop level j. This might introduce redundant computations. \\ \hline C.unroll(i, v) & Unroll the loop level i by a factor v. \\ \hline C.after(B, i) & Indicate that C should be ordered after B at the loop level i (they have the same order in all the loop levels above i). \\ \hline C.inline() & Inline C in all of its consumers. \\ \hline C.set_schedule() & Transform the iteration domain of C using an affine relation (a <i>map</i> to transform Layer I to II expressed in the ISL syntax). \\ \hline \multicolumn{2}{ c }{Commands for mapping loop levels to hardware} \\ \hline C.parallelize(i) & Parallelize the i loop level for execution on a shared memory system. \\ \hline C.vectorize(i, v) & Vectorize the loop level i by a vector size v. \\ \hline C.gpu(i_0, i_1, i_2, i_3) & Mark the loop levels i_0, i_1, i_2 and i_3 to be executed on GPU. (i_0, i_1) are mapped to block IDs and (i_2, i_3) to thread IDs. \\ \hline C.tile_gpu(i_0,i_1,t1,t2) & Tile the loops i_0 and i_1 by $t_1 \times t_2$ and map them to GPU. \\ \hline C.distribute(i) & Parallelize the loop level i for execution on a distributed memory system. \\ \hline \end{tabular}</pre>	(not rendered)	We assume that C and P are computations, b is a buffer i and j are loop iterators Commands for loop nest transformations <table><tr><th>Command</th><th>Description</th></tr><tr><td><code>C.tile(i,j,t1,t2, i0,j0,i1,j1)</code></td><td>Tile the loop levels (i, j) of the computation C by $t_1 \times t_2$. The names of the new loop levels are (i_0, j_0, i_1, j_1) where i_0 is the outermost loop level and j_1 is the innermost.</td></tr><tr><td><code>C.interchange(i, j)</code></td><td>Interchange the i and j loop levels of C.</td></tr><tr><td><code>C.shift(i, s)</code></td><td>Loop shifting (shift the loop level i by s iterations).</td></tr><tr><td><code>C.split(i, s, i0, i1)</code></td><td>Split the loop level i by s. $(i_0, i1)$ are the new loop levels.</td></tr><tr><td><code>P.compute_at(C, j)</code></td><td>Compute the computation P in the loop nest of C at loop level j. This might introduce redundant computations.</td></tr><tr><td><code>C.unroll(i, v)</code></td><td>Unroll the loop level i by a factor v.</td></tr><tr><td><code>C.after(B, i)</code></td><td>Indicate that C should be ordered after B at the loop level i (they have the same order in all the loop levels above i).</td></tr><tr><td><code>C.inline()</code></td><td>Inline C in all of its consumers.</td></tr><tr><td><code>C.set_schedule()</code></td><td>Transform the iteration domain of C using an affine relation (a <i>map</i> to transform Layer I to II expressed in the ISL syntax).</td></tr></table> Commands for mapping loop levels to hardware <table><tr><td><code>C.parallelize(i)</code></td><td>Parallelize the i loop level for execution on a shared memory system.</td></tr><tr><td><code>C.vectorize(i, v)</code></td><td>Vectorize the loop level i by a vector size v.</td></tr><tr><td><code>C.gpu(i0, i1, i2, i3)</code></td><td>Mark the loop levels $i_0, i1, i2$ and i_3 to be executed on GPU. $(i_0, i1)$ are mapped to block IDs and $(i2, i3)$ to thread IDs.</td></tr><tr><td><code>C.tile_gpu(i0,i1,t1,t2)</code></td><td>Tile the loops i_0 and $i1$ by $t1 \times t2$ and map them to GPU.</td></tr><tr><td><code>C.distribute(i)</code></td><td>Parallelize the loop level i for execution on a distributed memory system.</td></tr></table>	Command	Description	<code>C.tile(i,j,t1,t2, i0,j0,i1,j1)</code>	Tile the loop levels (i, j) of the computation C by $t_1 \times t_2$. The names of the new loop levels are (i_0, j_0, i_1, j_1) where i_0 is the outermost loop level and j_1 is the innermost.	<code>C.interchange(i, j)</code>	Interchange the i and j loop levels of C .	<code>C.shift(i, s)</code>	Loop shifting (shift the loop level i by s iterations).	<code>C.split(i, s, i0, i1)</code>	Split the loop level i by s . $(i_0, i1)$ are the new loop levels.	<code>P.compute_at(C, j)</code>	Compute the computation P in the loop nest of C at loop level j . This might introduce redundant computations.	<code>C.unroll(i, v)</code>	Unroll the loop level i by a factor v .	<code>C.after(B, i)</code>	Indicate that C should be ordered after B at the loop level i (they have the same order in all the loop levels above i).	<code>C.inline()</code>	Inline C in all of its consumers.	<code>C.set_schedule()</code>	Transform the iteration domain of C using an affine relation (a <i>map</i> to transform Layer I to II expressed in the ISL syntax).	<code>C.parallelize(i)</code>	Parallelize the i loop level for execution on a shared memory system.	<code>C.vectorize(i, v)</code>	Vectorize the loop level i by a vector size v .	<code>C.gpu(i0, i1, i2, i3)</code>	Mark the loop levels $i_0, i1, i2$ and i_3 to be executed on GPU. $(i_0, i1)$ are mapped to block IDs and $(i2, i3)$ to thread IDs.	<code>C.tile_gpu(i0,i1,t1,t2)</code>	Tile the loops i_0 and $i1$ by $t1 \times t2$ and map them to GPU.	<code>C.distribute(i)</code>	Parallelize the loop level i for execution on a distributed memory system.
Command	Description																																		
<code>C.tile(i,j,t1,t2, i0,j0,i1,j1)</code>	Tile the loop levels (i, j) of the computation C by $t_1 \times t_2$. The names of the new loop levels are (i_0, j_0, i_1, j_1) where i_0 is the outermost loop level and j_1 is the innermost.																																		
<code>C.interchange(i, j)</code>	Interchange the i and j loop levels of C .																																		
<code>C.shift(i, s)</code>	Loop shifting (shift the loop level i by s iterations).																																		
<code>C.split(i, s, i0, i1)</code>	Split the loop level i by s . $(i_0, i1)$ are the new loop levels.																																		
<code>P.compute_at(C, j)</code>	Compute the computation P in the loop nest of C at loop level j . This might introduce redundant computations.																																		
<code>C.unroll(i, v)</code>	Unroll the loop level i by a factor v .																																		
<code>C.after(B, i)</code>	Indicate that C should be ordered after B at the loop level i (they have the same order in all the loop levels above i).																																		
<code>C.inline()</code>	Inline C in all of its consumers.																																		
<code>C.set_schedule()</code>	Transform the iteration domain of C using an affine relation (a <i>map</i> to transform Layer I to II expressed in the ISL syntax).																																		
<code>C.parallelize(i)</code>	Parallelize the i loop level for execution on a shared memory system.																																		
<code>C.vectorize(i, v)</code>	Vectorize the loop level i by a vector size v .																																		
<code>C.gpu(i0, i1, i2, i3)</code>	Mark the loop levels $i_0, i1, i2$ and i_3 to be executed on GPU. $(i_0, i1)$ are mapped to block IDs and $(i2, i3)$ to thread IDs.																																		
<code>C.tile_gpu(i0,i1,t1,t2)</code>	Tile the loops i_0 and $i1$ by $t1 \times t2$ and map them to GPU.																																		
<code>C.distribute(i)</code>	Parallelize the loop level i for execution on a distributed memory system.																																		

Identifier	Page	Conf.	LaTeX source	Rendered	Scan image												
1804.10694v5_TAB_003	004	■ 0.726	$\begin{tabular}{t}{ l l } \hline \multicolumn{2}{ c } \{ \text{High level commands for data manipulation} \} \\ \hline C.\text{store_in}(b, \{i, j\}) \ \& \ \text{Store the result of the computation } \backslash(\mathrm{C}(\mathrm{i}, \mathrm{j})\backslash) \text{ in } \backslash(\mathrm{b}[\mathrm{i}], \mathrm{j})\backslash). \\ \hline C.\text{cache_shared_at}(P, i) \ \& \ \text{Cache (copy) the buffer of C in shared memory. Copying from global to shared GPU memory will be done at loop level i of the computation P. The amount of data to copy, the access functions, and synchronization are computed automatically.} \\ \hline C.\text{cache_local_at}(P, i) \ \& \ \text{Similar to cache_shared_at but stores in local GPU memory.} \\ \hline \text{send}(d, \text{src}, s, q, p) \ \& \ \text{Create a send operation. d: vector of iterators to represent the iteration domain of the send; src: source buffer; s: size; q: destination node; p: properties (synchronous, asynchronous, blocking, ...).} \\ \hline \text{receive}(d, \text{dst}, s, q, p) \ \& \ \text{Create a receive operation. Arguments similar to send except q, which is the source node.} \\ \hline \end{tabular}$	(not rendered)	<table><tr><th colspan="2">High level commands for data manipulation</th></tr><tr><td>$C.\text{store_in}(b, \{i, j\})$</td><td>Store the result of the computation $C(i,j)$ in $b[i,j]$.</td></tr><tr><td>$C.\text{cache_shared_at}(P, i)$</td><td>Cache (copy) the buffer of C in shared memory. Copying from global to shared GPU memory will be done at loop level i of the computation P. The amount of data to copy, the access functions, and synchronization are computed automatically.</td></tr><tr><td>$C.\text{cache_local_at}(P, i)$</td><td>Similar to cache_shared_at but stores in local GPU memory.</td></tr><tr><td>$\text{send}(d, \text{src}, s, q, p)$</td><td>Create a send operation. d: vector of iterators to represent the iteration domain of the send; src: source buffer; s: size; q: destination node; p: properties (synchronous, asynchronous, blocking, ...).</td></tr><tr><td>$\text{receive}(d, \text{dst}, s, q, p)$</td><td>Create a receive operation. Arguments similar to send except q, which is the source node.</td></tr></table>	High level commands for data manipulation		$C.\text{store_in}(b, \{i, j\})$	Store the result of the computation $C(i,j)$ in $b[i,j]$.	$C.\text{cache_shared_at}(P, i)$	Cache (copy) the buffer of C in shared memory. Copying from global to shared GPU memory will be done at loop level i of the computation P . The amount of data to copy, the access functions, and synchronization are computed automatically.	$C.\text{cache_local_at}(P, i)$	Similar to cache_shared_at but stores in local GPU memory.	$\text{send}(d, \text{src}, s, q, p)$	Create a send operation. d : vector of iterators to represent the iteration domain of the send; src : source buffer; s : size; q : destination node; p : properties (synchronous, asynchronous, blocking, ...).	$\text{receive}(d, \text{dst}, s, q, p)$	Create a receive operation. Arguments similar to send except q , which is the source node.
High level commands for data manipulation																	
$C.\text{store_in}(b, \{i, j\})$	Store the result of the computation $C(i,j)$ in $b[i,j]$.																
$C.\text{cache_shared_at}(P, i)$	Cache (copy) the buffer of C in shared memory. Copying from global to shared GPU memory will be done at loop level i of the computation P . The amount of data to copy, the access functions, and synchronization are computed automatically.																
$C.\text{cache_local_at}(P, i)$	Similar to cache_shared_at but stores in local GPU memory.																
$\text{send}(d, \text{src}, s, q, p)$	Create a send operation. d : vector of iterators to represent the iteration domain of the send; src : source buffer; s : size; q : destination node; p : properties (synchronous, asynchronous, blocking, ...).																
$\text{receive}(d, \text{dst}, s, q, p)$	Create a receive operation. Arguments similar to send except q , which is the source node.																

Conf. MathPix confidence:

■ ≥0.9

■ 0.5–0.9

■ <0.5

— no value

LaTeX source MathPix’s reading, unchanged; Rendered the same reading, with a display environment (align/alignat/flalign/eqnarray/gather/multline/split) mapped to its in-math form so it can typeset in this cell — the two may legitimately differ

Residual render vs scan (inkdrill):

● C component

● W weak

● S stable

● N noise

● K clean

● X crop overran its region

● not measured

[refined: basis] beside an identifier: the LaTeX-source and Rendered cells both show a VERIFIED refinement, not MathPix’s own reading — basis names what verified it (e.g. ink, source). The original reading is unchanged in the model but is not the one drawn here.

Identifier	Page	Conf.	LaTeX source	Rendered	Scan image																						
1804.10694v5_TAB_004	004	■0.630	\begin{tabular}[t]{l l } \hline \multicolumn{2}{ c } {Low level commands for data manipulation} \\ \hline Buffer b(sizes, type) & Declare a buffer (sizes: a vector of dimension sizes). \\ \hline b.allocate_at (p, i) & Return an operation that allocates b at the loop i of p . An operation can be scheduled like any computation. \\ \hline C.buffer() & Return the buffer associated to the computation C. \\ \hline b.set_size(sizes) & Set the size of a buffer. sizes: a vector of dimension sizes. \\ \hline b.tag_gpu_global() & Tag buffer to be stored in global GPU memory. \\ \hline b.tag_gpu_shared() & Tag buffer to be stored in shared GPU memory. \\ \hline b.tag_gpu_local() & Tag buffer to be stored in local GPU memory. \\ \hline b.tag_gpu_constant() & Tag buffer to be stored in constant GPU memory. \\ \hline C.host_to_device () & Return an operation that copies C.buffer() from host to device. \\ \hline C.device_to_host () & Return an operation that copies C.buffer() from device to host. \\ \hline copy_at (p, i, bs, bd) & Return an operation that copies the buffer bs to the buffer bd at the loop i of p . Used for copies between global, shared and local. \\ \hline \end{tabular}	(not rendered)	Low level commands for data manipulation <table><tr><td>Buffer b(sizes, type)</td><td>Declare a buffer (sizes: a vector of dimension sizes).</td></tr><tr><td>b.allocate_at(p, i)</td><td>Return an operation that allocates b at the loop i of p. An operation can be scheduled like any computation.</td></tr><tr><td>C.buffer()</td><td>Return the buffer associated to the computation C.</td></tr><tr><td>b.set_size(sizes)</td><td>Set the size of a buffer. sizes: a vector of dimension sizes.</td></tr><tr><td>b.tag_gpu_global()</td><td>Tag buffer to be stored in global GPU memory.</td></tr><tr><td>b.tag_gpu_shared()</td><td>Tag buffer to be stored in shared GPU memory.</td></tr><tr><td>b.tag_gpu_local()</td><td>Tag buffer to be stored in local GPU memory.</td></tr><tr><td>b.tag_gpu_constant()</td><td>Tag buffer to be stored in constant GPU memory.</td></tr><tr><td>C.host_to_device()</td><td>Return an operation that copies C.buffer() from host to device.</td></tr><tr><td>C.device_to_host()</td><td>Return an operation that copies C.buffer() from device to host.</td></tr><tr><td>copy_at(p, i, bs, bd)</td><td>Return an operation that copies the buffer bs to the buffer bd at the loop i of p. Used for copies between global, shared and local.</td></tr></table>	Buffer b(sizes, type)	Declare a buffer (sizes: a vector of dimension sizes).	b.allocate_at(p, i)	Return an operation that allocates b at the loop i of p. An operation can be scheduled like any computation.	C.buffer()	Return the buffer associated to the computation C.	b.set_size(sizes)	Set the size of a buffer. sizes: a vector of dimension sizes.	b.tag_gpu_global()	Tag buffer to be stored in global GPU memory.	b.tag_gpu_shared()	Tag buffer to be stored in shared GPU memory.	b.tag_gpu_local()	Tag buffer to be stored in local GPU memory.	b.tag_gpu_constant()	Tag buffer to be stored in constant GPU memory.	C.host_to_device()	Return an operation that copies C.buffer() from host to device.	C.device_to_host()	Return an operation that copies C.buffer() from device to host.	copy_at(p, i, bs, bd)	Return an operation that copies the buffer bs to the buffer bd at the loop i of p. Used for copies between global, shared and local.
					Buffer b(sizes, type)	Declare a buffer (sizes: a vector of dimension sizes).																					
b.allocate_at(p, i)	Return an operation that allocates b at the loop i of p. An operation can be scheduled like any computation.																										
C.buffer()	Return the buffer associated to the computation C.																										
b.set_size(sizes)	Set the size of a buffer. sizes: a vector of dimension sizes.																										
b.tag_gpu_global()	Tag buffer to be stored in global GPU memory.																										
b.tag_gpu_shared()	Tag buffer to be stored in shared GPU memory.																										
b.tag_gpu_local()	Tag buffer to be stored in local GPU memory.																										
b.tag_gpu_constant()	Tag buffer to be stored in constant GPU memory.																										
C.host_to_device()	Return an operation that copies C.buffer() from host to device.																										
C.device_to_host()	Return an operation that copies C.buffer() from device to host.																										
copy_at(p, i, bs, bd)	Return an operation that copies the buffer bs to the buffer bd at the loop i of p. Used for copies between global, shared and local.																										
1804.10694v5_TAB_005	004	■0.999	\begin{tabular}[t]{l l } \hline \multicolumn{2}{ c }{Commands for synchronization} \\ \hline barrier_at (p, i) & Create a barrier at the loop p of i. \\ \hline \end{tabular}	(not rendered)	Commands for synchronization <table><tr><td>barrier_at(p, i)</td><td>Create a barrier at the loop p of i.</td></tr></table>	barrier_at(p, i)	Create a barrier at the loop p of i.																				
barrier_at(p, i)	Create a barrier at the loop p of i.																										

Conf. MathPix confidence:

■ ≥0.9

■ 0.5-0.9

■ <0.5

 — no value

LaTeX source MathPix’s reading, unchanged; Rendered the same reading, with a display environment (align/alignat/flalign/eqnarray/gather/multline/split) mapped to its in-math form so it can typeset in this cell — the two may legitimately differ

Residual render vs scan (inkdrill):

● C component

● W weak

● S stable

● N noise

● K clean

● X crop overran its region

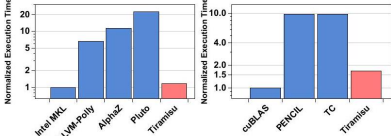
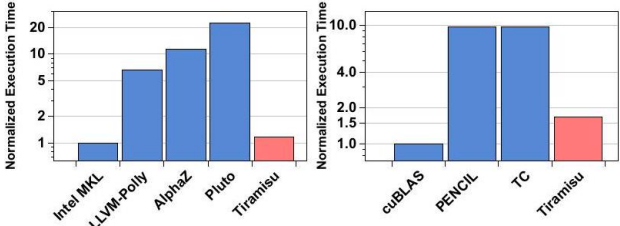
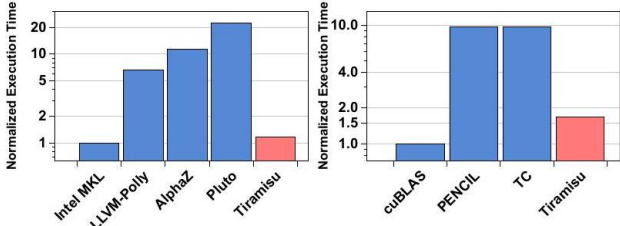
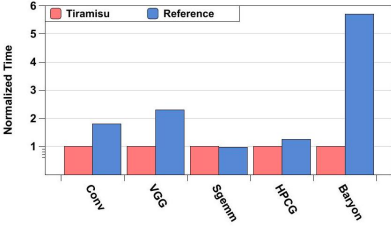
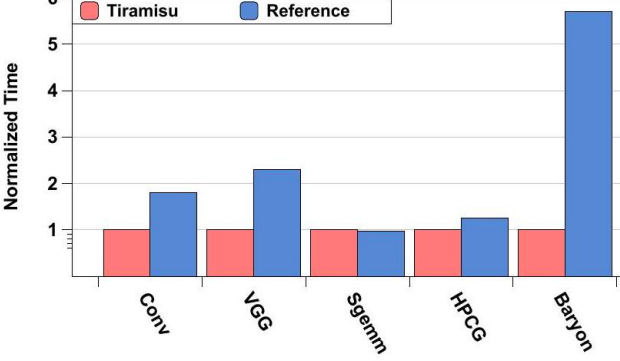
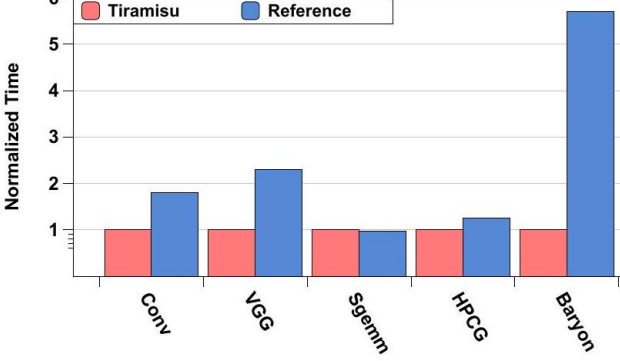
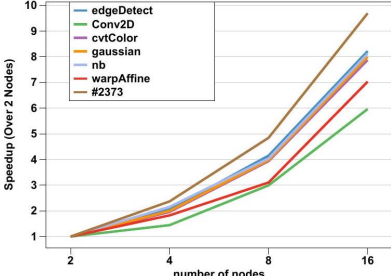
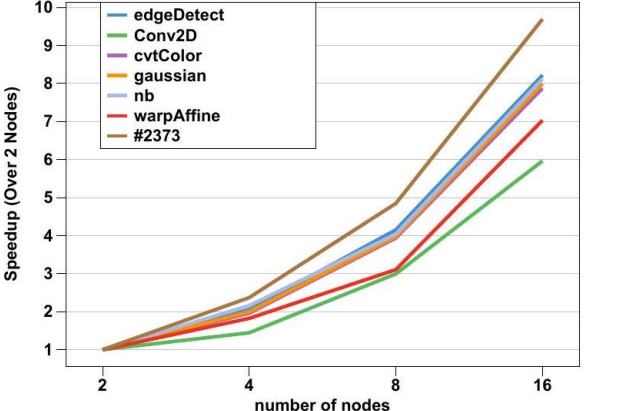
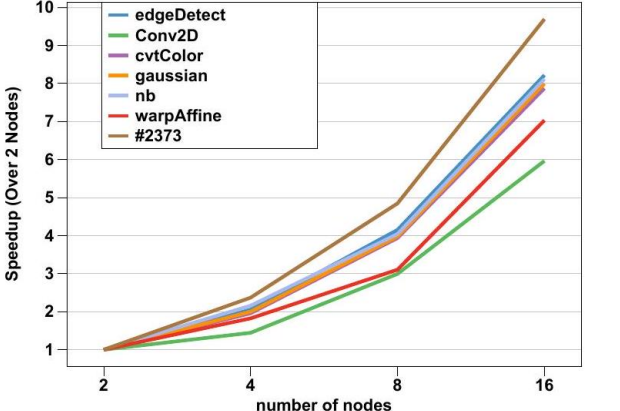
● not measured

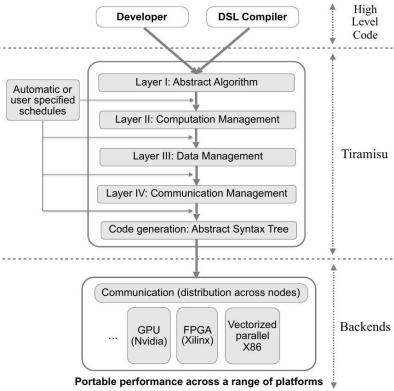
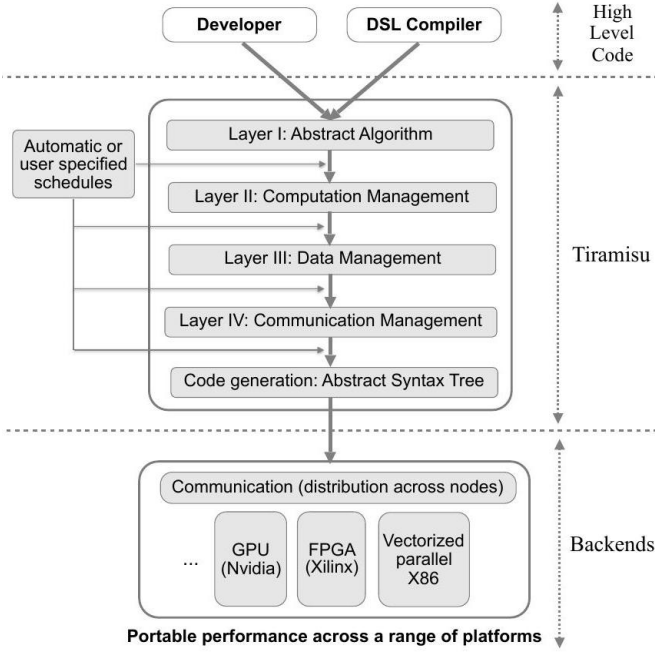
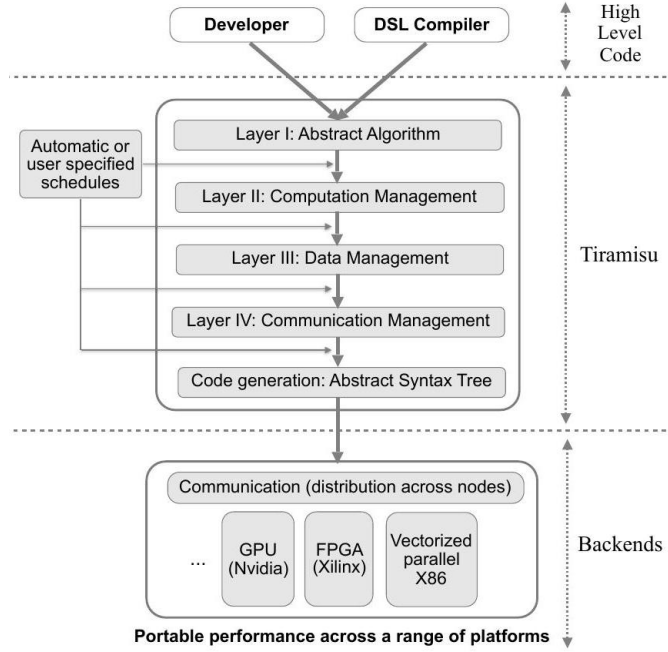
[refined: basis] beside an identifier: the LaTeX-source and Rendered cells both show a VERIFIED refinement, not MathPix’s own reading — basis names what verified it (e.g. ink, source). The original reading is unchanged in the model but is not the one drawn here.

Identifier	Page	Conf.	LaTeX source	Rendered	Scan image																																																																																																																
1804.10694v5_TAB_006	011	■1.000	\begin{tabular}[t]{ l l l l l l l l l }\hline Architectures & \multicolumn{8}{ l }{Frameworks} \\ \hline & & edge Detector & cvtColor & Conv2D & warp Affine & gaussian & nb & ticket #2373 \\ \hline & & & & & & & & \\ \hline Single-node multicore & Tiramisu & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ & Halide & - & 1 & 1 & 1 & 1 & 3.77 & - \\ & PENCIL & 2.43 & 2.39 & 11.82 & 10.2 & 5.82 & 1 & 1 \\ \hline & & & & & & & & \\ \hline GPU & Tiramisu & 1.05 & 1 & 1 & 1 & 1 & 1 & 1 \\ & Halide & - & 1 & 1.3 & 1 & 1.3 & 1.7 & - \\ & PENCIL & 1 & 1 & 1.33 & 1 & 1.2 & 1.02 & 1 \\ \hline & & & & & & & & \\ \hline Distributed (16 Nodes) & Tiramisu & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ & Dist-Halide & - & 1.31 & 3.25 & 2.54 & 1.57 & 1.45 & - \\ \hline \end{tabular}	(not rendered)	<table><tr><th>Architectures</th><th>Frameworks</th><th colspan="7">Benchmarks</th></tr><tr><td></td><td></td><td>edge Detector</td><td>cvtColor</td><td>Conv2D</td><td>warp Affine</td><td>gaussian</td><td>nb</td><td>ticket #2373</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td rowspan="3">Single-node multicore</td><td>Tiramisu</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>Halide</td><td>-</td><td>1</td><td>1</td><td>1</td><td>1</td><td>3.77</td><td>-</td></tr><tr><td>PENCIL</td><td>2.43</td><td>2.39</td><td>11.82</td><td>10.2</td><td>5.82</td><td>1</td><td>1</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td rowspan="3">GPU</td><td>Tiramisu</td><td>1.05</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>Halide</td><td>-</td><td>1</td><td>1.3</td><td>1</td><td>1.3</td><td>1.7</td><td>-</td></tr><tr><td>PENCIL</td><td>1</td><td>1</td><td>1.33</td><td>1</td><td>1.2</td><td>1.02</td><td>1</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td rowspan="2">Distributed (16 Nodes)</td><td>Tiramisu</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>Dist-Halide</td><td>-</td><td>1.31</td><td>3.25</td><td>2.54</td><td>1.57</td><td>1.45</td><td>-</td></tr></table>	Architectures	Frameworks	Benchmarks									edge Detector	cvtColor	Conv2D	warp Affine	gaussian	nb	ticket #2373										Single-node multicore	Tiramisu	1	1	1	1	1	1	1	Halide	-	1	1	1	1	3.77	-	PENCIL	2.43	2.39	11.82	10.2	5.82	1	1										GPU	Tiramisu	1.05	1	1	1	1	1	1	Halide	-	1	1.3	1	1.3	1.7	-	PENCIL	1	1	1.33	1	1.2	1.02	1										Distributed (16 Nodes)	Tiramisu	1	1	1	1	1	1	1	Dist-Halide	-	1.31	3.25	2.54	1.57	1.45	-
					Architectures	Frameworks	Benchmarks																																																																																																														
		edge Detector	cvtColor	Conv2D	warp Affine	gaussian	nb	ticket #2373																																																																																																													
Single-node multicore	Tiramisu	1	1	1	1	1	1	1																																																																																																													
	Halide	-	1	1	1	1	3.77	-																																																																																																													
	PENCIL	2.43	2.39	11.82	10.2	5.82	1	1																																																																																																													
GPU	Tiramisu	1.05	1	1	1	1	1	1																																																																																																													
	Halide	-	1	1.3	1	1.3	1.7	-																																																																																																													
	PENCIL	1	1	1.33	1	1.2	1.02	1																																																																																																													
Distributed (16 Nodes)	Tiramisu	1	1	1	1	1	1	1																																																																																																													
	Dist-Halide	-	1.31	3.25	2.54	1.57	1.45	-																																																																																																													
Conf. MathPix confidence: ■ ≥0.9 ■ 0.5-0.9 ■ <0.5 — no value LaTeX source MathPix’s reading, unchanged; Rendered the same reading, with a display environment (align/alignat/flalign/eqnarray/gather/multline/split) mapped to its in-math form so it can typeset in this cell — the two may legitimately differ Residual render vs scan (inkdrill): ● C component ● W weak ● S stable ● N noise ● K clean ● X crop overran its region ● not measured [refined: basis] beside an identifier: the LaTeX-source and Rendered cells both show a VERIFIED refinement, not MathPix’s own reading — basis names what verified it (e.g. ink, source). The original reading is unchanged in the model but is not the one drawn here.																																																																																																																					

Image regions — rendered against scan

Two comparable cells per row. **Rendered**: the region’s own LaTeX compiled as its OWN document, or — where no LaTeX exists — the scan again, so every row has two cells and the ink difference reads as a floor rather than as missing-versus-present. A duplicated row is marked *(dup)* and its distance is a SELF-comparison, not agreement between two sources. **Scan**: the tex.zip image whose filename region 5-tuple matches this row, else the CDN crop. The Class column carries the residual class beside MathPix’s confidence, as the equation rows do. Setting a region’s LaTeX does NOT say it renders to what is on the page — that is what the two columns are for, and 281 is the open question they exist to let a reader answer. A region with no LaTeX can be reconstructed with pdfdrill vision; verify any LLM result against the real ink with inkdrill.

Identifier	Page	Class	Source	Author source	Rendered	Scan
1804.10694v5_PIC_0001	001	— (dup)	c195a42a-b20c-41a1-bc53-3496defca0ba-01_309_814_834_1111.jpg			
1804.10694v5_PIC_0002	009	— (dup)	c195a42a-b20c-41a1-bc53-3496defca0ba-09_488_844_178_1085.jpg			
1804.10694v5_PIC_0003	011	— (dup)	c195a42a-b20c-41a1-bc53-3496defca0ba-11_548_761_885_1121.jpg			
1804.10694v5_DIA_0001	003	— (dup)	no image for this region (4 in zip)	(no tex.zip crop for this row)	<pre>1 // Declare the iterators i, j and c. 2 Var i(0, N-2), j(0, M-2), c(0, 3); 3 4 Computation bx(i, j, c), by(i, j, c); 5 6 // Algorithm. 7 bx(i, j, c) = (in(i, j, c)+in(i, j+1, c)+in(i, j+2, c))/3; 8 by(i, j, c) = (bx(i, j, c)+bx(i+1, j, c)+bx(i+2, j, c))/3;</pre>	<pre>1 // Declare the iterators i, j and c. 2 Var i(0, N-2), j(0, M-2), c(0, 3); 3 4 Computation bx(i, j, c), by(i, j, c); 5 6 // Algorithm. 7 bx(i, j, c) = (in(i, j, c)+in(i, j+1, c)+in(i, j+2, c))/3; 8 by(i, j, c) = (bx(i, j, c)+bx(i+1, j, c)+bx(i+2, j, c))/3;</pre>
1804.10694v5_DIA_0002	004	— (dup)	no image for this region (4 in zip)	(no tex.zip crop for this row)	<pre>for (i in 0..N-2) for (j in 0..M-2) for (c in 0..3) bx[i][j][c] = (in[i][j][c]+in[i][j+1][c]+in[i][j+2][c])/3 for (i in 0..N-2) for (j in 0..M-2) for (c in 0..3) by[i][j][c] = (bx[i][j][c]+bx[i+1][j][c]+bx[i+2][j][c])/3</pre>	<pre>for (i in 0..N-2) for (j in 0..M-2) for (c in 0..3) bx[i][j][c] = (in[i][j][c]+in[i][j+1][c]+in[i][j+2][c])/3 for (i in 0..N-2) for (j in 0..M-2) for (c in 0..3) by[i][j][c] = (bx[i][j][c]+bx[i+1][j][c]+bx[i+2][j][c])/3</pre>
1804.10694v5_DIA_0003	005	— (dup)	no image for this region (4 in zip)	(no tex.zip crop for this row)	<pre>1 // Scheduling commands for targeting 2 // a multicore architecture. 3</pre>	<pre>1 // Scheduling commands for targeting 2 // a multicore architecture. 3</pre>

1804.10694v5_DIA_0004	005	— (dup)	no image for this region (4 in zip)	(no tex.zip crop for this row)	<pre> Parallel for(i0 in 0..floor((N-2)/32)) for(j0 in 0..floor((M-2)/32)) bx[32,34,3]; // Tiling with redundancy for(i1 in 0..min((N-2)%32,32)+2) for(j1 in 0..min((M-2)%32,32)+2) int i = i0*32+i1 int j = j0*32+j1 for(c in 0..3) bx[i1][j1][c]= (in[i][j][c] + in[i][j+1][c] + in[i][j+2][c])/3 for(i1 in 0..min(N-2,32)) for(j1 in 0..min(M-2,32)) int i = i0*32+i1 int j = j0*32+j1 for(c in 0..3) by[i][j][c]= (bx[i][j][c] + bx[i+1][j][c] + bx[i+2][j][c])/3 </pre>	<pre> Parallel for(i0 in 0..floor((N-2)/32)) for(j0 in 0..floor((M-2)/32)) bx[32,34,3]; // Tiling with redundancy for(i1 in 0..min((N-2)%32,32)+2) for(j1 in 0..min((M-2)%32,32)+2) int i = i0*32+i1 int j = j0*32+j1 for(c in 0..3) bx[i1][j1][c]= (in[i][j][c] + in[i][j+1][c] + in[i][j+2][c])/3 for(i1 in 0..min(N-2,32)) for(j1 in 0..min(M-2,32)) int i = i0*32+i1 int j = j0*32+j1 for(c in 0..3) by[i][j][c]= (bx[i][j][c] + bx[i+1][j][c] + bx[i+2][j][c])/3 </pre>
1804.10694v5_DIA_0005	005	— (dup)	no image for this region (4 in zip)	(no tex.zip crop for this row)	<pre> // We assume that in[][][] is initially // distributed across nodes. Each node // has a chunk of the original // in[][][] that we call lin[][][]. // Start by exchanging border rows of // lin[][][] distributed for (is in 1..Nodes) send(lin(0,0,0), M*2*3, is-1, {ASYNC}) distributed for (ir in 0..Nodes-1) recv(lin(N,0,0), M*2*3, ir+1, {SYNC}) distributed for (i0 in 0..Nodes) parallel for (i1 in 0..(N-2)/Nodes) int i = i0*((N-2)/Nodes) + i1 for (j in 0..M-2) for (c in 0..3) bx[i][j][c] = (lin[i][j][c] + lin[i][j+1][c] + lin[i][j+2][c])/3 distributed for (i0 in 0..Nodes) parallel for (i1 in 0..(N-2)/Nodes) int i = q*((N-2)/Nodes) + i1 for (j in 0..M-2) for (c in 0..3) by[i][j][c] = (bx[i][j][c] + bx[i+1][j][c] + bx[i+2][j][c])/3 // We assume that no gather operation on // by[][][] is needed </pre>	<pre> // We assume that in[][][] is initially // distributed across nodes. Each node // has a chunk of the original // in[][][] that we call lin[][][]. // Start by exchanging border rows of // lin[][][] distributed for (is in 1..Nodes) send(lin(0,0,0), M*2*3, is-1, {ASYNC}) distributed for (ir in 0..Nodes-1) recv(lin(N,0,0), M*2*3, ir+1, {SYNC}) distributed for (i0 in 0..Nodes) parallel for (i1 in 0..(N-2)/Nodes) int i = i0*((N-2)/Nodes) + i1 for (j in 0..M-2) for (c in 0..3) bx[i][j][c] = (lin[i][j][c] + lin[i][j+1][c] + lin[i][j+2][c])/3 distributed for (i0 in 0..Nodes) parallel for (i1 in 0..(N-2)/Nodes) int i = q*((N-2)/Nodes) + i1 for (j in 0..M-2) for (c in 0..3) by[i][j][c] = (bx[i][j][c] + bx[i+1][j][c] + bx[i+2][j][c])/3 // We assume that no gather operation on // by[][][] is needed </pre>
1804.10694v5_DIA_0006	007	— (dup)	c195a42a-b20c-41a1-bc53-3496defca0ba-07_849_859_176_1083.jpg	 Portable performance across a range of platforms	 Portable performance across a range of platforms	 Portable performance across a range of platforms
1804.10694v5_DIA_0007	009	— (dup)	no image for this region (4 in zip)	(no tex.zip crop for this row)	for(q in 1..N-1) {...} // distribute on q	for(q in 1..N-1) {...} // distribute on q
1804.10694v5_DIA_0008	009	— (dup)	no image for this region (4 in zip)	(no tex.zip crop for this row)	q = get_rank(); if (q≥1 and q<N-1) {...}	q = get_rank(); if (q≥1 and q<N-1) {...}

1804.10694v5_D1A_0009	010	— (dup)	no image for this region (4 in zip)	(no tex.zip crop for this row)	<pre>/* Ring Blur Filter */ R(i,j) = (Img(i-1,j-1) + Img(i-1,j) + Img(i-1,j+1) + Img(i,j-1) + Img(i,j+1) + Img(i+1,j-1) + Img(i+1,j) + Img(i+1,j+1))/8 /* Roberts Edge Detection Filter */ Img(i,j) = abs(R(i,j) - R(i+1,j-1)) + abs(R(i+1,j) - R(i,j-1))</pre>	<pre>/* Ring Blur Filter */ R(i,j) = (Img(i-1,j-1) + Img(i-1,j) + Img(i-1,j+1) + Img(i,j-1) + Img(i,j+1) + Img(i+1,j-1) + Img(i+1,j) + Img(i+1,j+1))/8 /* Roberts Edge Detection Filter */ Img(i,j) = abs(R(i,j) - R(i+1,j-1)) + abs(R(i+1,j) - R(i,j-1))</pre>
-----------------------	-----	---------	-------------------------------------	--------------------------------	--	--